

Fee Management System Using Microsoft Azure

Deployment Guide

1. Project Overview

The Fee Management System is a cloud-native, enterprise-grade application developed using Microsoft Azure services and Node.js (Azure Functions v4 Programming Model). The solution enables students to check real-time fee payment status and outstanding balances, provides administrators with a secure interface to update student fee payments, and automatically dispatches personalized email reminders to students with overdue fees.

The system leverages Node.js Azure Functions for high-performance serverless APIs, Azure SQL Database for persistent relational storage, Azure API Management (APIM) for API gateway routing, rate-limiting, and security enforcement, Microsoft Entra ID (Azure AD) for OAuth 2.0 authentication and Role-Based Access Control (RBAC), Azure Logic Apps for scheduled workflow automation, a responsive Frontend Web Dashboard for user interaction, and Azure Application Insights for end-to-end monitoring and live telemetry diagnostics.

2. Technologies Used

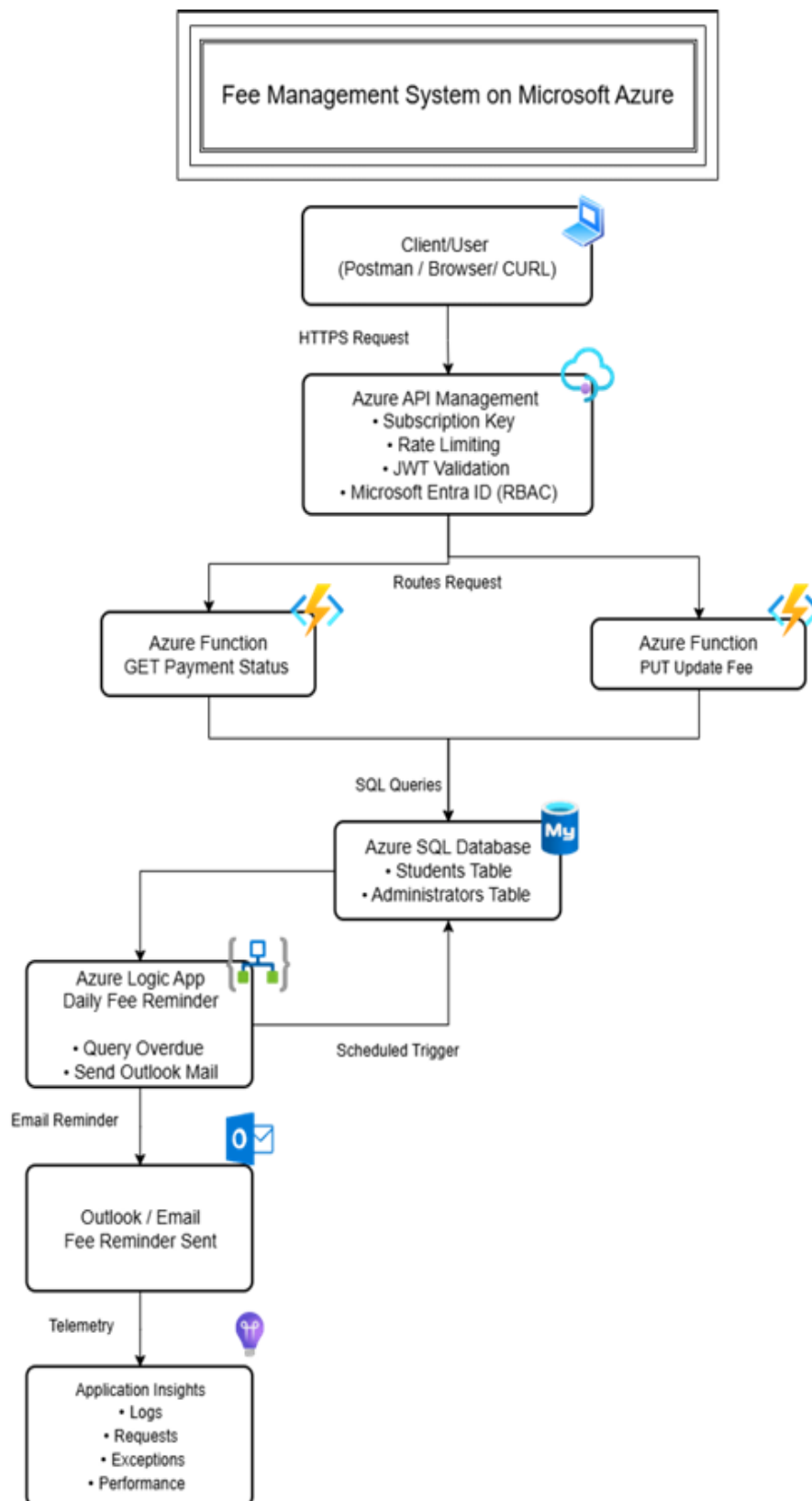
Programming & Web Technologies

- Node.js (v18+ / v20+ LTS)
- JavaScript (ES6+)
- SQL (Transact-SQL)
- HTML5 & CSS3 (Frontend Web Dashboard)

Microsoft Azure Services

- Azure SQL Database
- Azure SQL Server
- Azure Functions (Node.js v4 Programming Model)
- Azure API Management (APIM)
- Azure Logic Apps
- Microsoft Entra ID (Azure AD)
- Azure Application Insights
- Azure Storage Account

Project Architecture Diagram



3. Required Software Installation

Install the following software before starting the project:

Software / Tool	Purpose
Node.js (v18.x or v20.x LTS)	Backend JavaScript Runtime
npm (Node Package Manager)	Package & Dependency Management
Visual Studio Code	Integrated Development Environment (IDE)
Git	Version Control
Azure CLI	Azure Cloud Resource Management
Azure Functions Core Tools (v4)	Local Function Execution & Deployment
SQL Server (MSSQL) VS Code Extension	Database Management & Query Execution
Azure Tools / Azure Functions Extension	Azure Functions & Cloud Deployment
Live Server VS Code Extension	Local Frontend Dashboard Hosting

4. Installation and Environment Setup

Step 1: Install Node.js & npm

Download and install Node.js (v18.x or v20.x LTS) from <https://nodejs.org/>.

Verify installation:

```
bash
```

```
node -v
```

```
npm -v
```

Step 2: Install Git

- Windows: Download and install from <https://git-scm.com/>
- Ubuntu / WSL:

```
bash
```

```
sudo apt update
```

```
sudo apt install git
```

Verify:

bash

git --version

Step 3: Install Visual Studio Code

Download and install Visual Studio Code from <https://code.visualstudio.com/>.

Step 4: Install Required VS Code Extensions

Open VS Code (Ctrl + Shift + X) and install the following extensions:

- Azure Functions (ms-azuretools.vscode-azurefunctions)
 - Azure Resources (ms-azuretools.vscode-azureresourcegroups)
 - SQL Server (mssql) (ms-mssql.mssql)
 - Live Server (ritwickdey.liveserver)
-

Step 5: Install Azure CLI

- **Windows (PowerShell):**

powershell

winget install Microsoft.AzureCLI

- Ubuntu / WSL:

bash

curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

Verify & Login:

bash

az version

az login

Step 6: Install Azure Functions Core Tools (v4)

Install globally via npm:

bash

npm install -g azure-functions-core-tools@4 --unsafe-perm true

Verify:

bash

func --version

Step 7: Clone the Repository

bash

```
git clone <REPOSITORY_URL>
cd fee-management-system
```

Step 8: Install Node.js Dependencies

bash

```
npm install
```

Step 9: Create local.settings.json

Create a file named local.settings.json in the root folder:

json

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "node",
    "DB_SERVER": "sqlserver-fee-mgmt.database.windows.net",
    "DB_NAME": "fee-management-db",
    "DB_USER": "sqladmin",
    "DB_PASSWORD": "<YOUR_SQL_PASSWORD>",
    "APPLICATIONINSIGHTS_CONNECTION_STRING":
"<YOUR_APP_INSIGHTS_CONNECTION_STRING>"
  }
}
```

Step 10: Run the Azure Function Locally

bash

```
func start
```

Step 11: Launch Frontend Dashboard Locally

Open Frontend/index.html in your browser or run with Live Server / static server:

bash

cd Frontend

python -m http.server 5500

Step 12: Deploy to Azure

bash

az login

func azure functionapp publish <FUNCTION_APP_NAME>

5. Create Azure Resources

Create the following Azure resources in the given order:

1. Resource Group (e.g., fee-management)
 2. Azure SQL Server (e.g., sqlserver-fee-mgmt)
 3. Azure SQL Database (e.g., fee-management-db)
 4. Azure Storage Account (e.g., stfeemgmt)
 5. Azure Application Insights (e.g., appi-fee-mgmt)
 6. Azure Function App (Runtime: Node.js 18 or 20 LTS, OS: Linux/Windows Consumption)
 7. Azure API Management (Consumption or Developer Tier)
 8. Microsoft Entra ID App Registrations (API Application & Client Application)
 9. Azure Logic App (Consumption Workflow)
-

6. Database Setup

Step 1: Connect to Azure SQL Database

Connect via Azure Portal Query Editor or Visual Studio Code using the SQL Server (mssql) extension.

Connection Details:

- Server Name: sqlserver-fee-mgmt.database.windows.net
- Database Name: fee-management-db
- Authentication: SQL Server Authentication
- Username: sqladmin
- Password: <YOUR_SQL_PASSWORD>

Step 2: Execute Database Setup Script

Execute the provided SQL script:
database/database_scripts.txt

The script performs the following operations:

- Creates the Students table (stores StudentID, Name, Email, Course, TotalFee, PaidAmount, DueDate).
 - Creates the Administrators table (stores AdminID, Name, Role).
 - Inserts sample student records with diverse fee and due date statuses.
 - Inserts sample administrator records.
 - Contains parameterized queries used by Azure Functions and Logic Apps.
-

7. Azure Function Setup (Node.js)

Step 1: Clone and Navigate to the Project

```
git clone <repository-url>
```

```
cd fee-management-system
```

Step 2: Install Node.js Dependencies

```
bash
```

```
npm install
```

Step 3: Configure local.settings.json

Create a file named local.settings.json in the root folder:

json

```
{  
  "IsEncrypted": false,  
  "Values": {  
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",  
    "FUNCTIONS_WORKER_RUNTIME": "node",  
    "DB_SERVER": "sqlserver-fee-mgmt.database.windows.net",  
    "DB_NAME": "fee-management-db",  
    "DB_USER": "sqladmin",  
    "DB_PASSWORD": "<YOUR_SQL_PASSWORD>",  
    "APPLICATIONINSIGHTS_CONNECTION_STRING":  
    "<YOUR_APP_INSIGHTS_CONNECTION_STRING>"  
  }  
}
```

Step 4: Run Function Locally

bash

func start

Step 5: Deploy to Azure

bash

az login

func azure functionapp publish <FUNCTION_APP_NAME>

8. API Management Configuration

Import the deployed Node.js Azure Function App into Azure API Management (APIM).

Configured API Operations:

1. GET /paymentstatus/{studentid}

- Purpose: Retrieves real-time student details, fee information, and payment status.
- Returns:
 - Student Details (StudentID, Name)
 - Fee Details (TotalFee, PaidAmount, DueDate, OutstandingAmount)
 - Payment Status (Paid, Overdue, Partially Paid)

2. PUT /updatefee/{studentid}

- Purpose: Allows authorized administrators to update the total paid amount for a student.
- Request Body:

json

```
{  
  "PaidAmount": <AMOUNT>  
}
```

Configure Subscription Key Authentication

- Setting: Subscription required enabled on the API Product.
 - Header Name: Ocp-Apim-Subscription-Key
 - Generate:
 - Primary Subscription Key
 - Secondary Subscription Key
-

Configure Rate Limiting Policy

Add inbound rate-limiting policy on the GET /paymentstatus/{studentid} operation:

- Threshold: 10 requests per 60 seconds.
- Policy XML (APIM/payment-status-api-policy.xml):

xml

```
<inbound>
  <base />
  <rate-limit calls="10" renewal-period="60" />
</inbound>
```

Configure JWT Validation Policy

Add inbound JWT token validation on the PUT /updatefee/{studentid} operation:

- Policy XML (APIM/update-fee-api-policy.xml):

xml

```
<inbound>
  <base />
  <validate-jwt header-name="Authorization" failed-validation-httpcode="401" failed-
validation-error-message="Unauthorized - Valid Microsoft Entra token required" require-
scheme="Bearer">
    <openid-config url="https://login.microsoftonline.com/{TENANT_ID}/v2.0/.well-
known/openid-configuration" />
    <audiences>
      <audience>{CLIENT_ID}</audience>
    </audiences>
    <required-claims>
      <claim name="roles" match="any">
        <value>Admin</value>
      </claim>
    </required-claims>
  </validate-jwt>
</inbound>
```

9. Microsoft Entra ID (Azure AD) Configuration

Step 1: Register API Application

- Create an App Registration named fee-management-admin-api.
- Note the Application (Client) ID and Directory (Tenant) ID.

Step 2: Expose an API

- Set Application ID URI: api://<CLIENT_ID>
- Add scope: api://<CLIENT_ID>/.default

Step 3: Create App Role

- Role Name: Admin
- Allowed Member Types: Applications
- Value: Admin
- Description: Administrative role with fee modification privileges

Step 4: Register Client Application

- Create an App Registration named fee-management-api-client.
- Under Certificates & secrets, generate a new Client Secret (copy the Value).

Step 5: Grant API Permissions & Admin Consent

- Under API permissions → Add a permission → My APIs → Select fee-management-admin-api.
- Select the Admin Application permission.
- Click Grant admin consent for Directory.

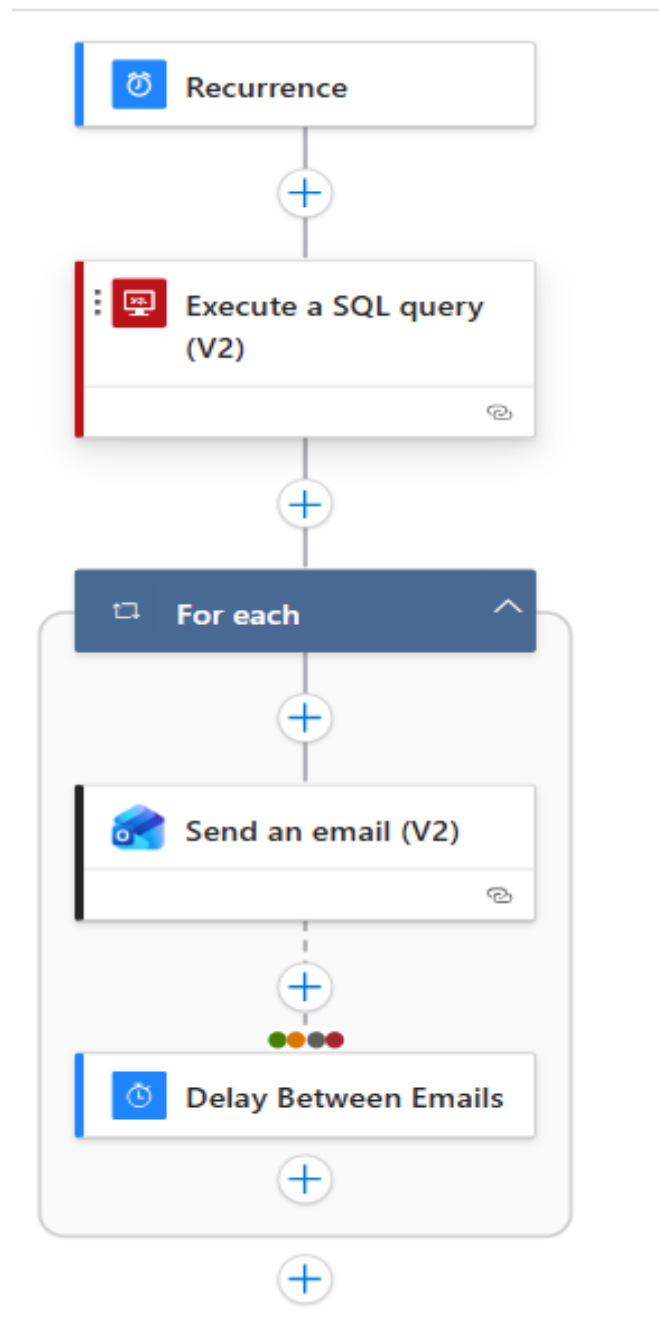
Step 6: Configure Role-Based Access Control (RBAC)

- The client application now receives the roles: ["Admin"] claim in its OAuth 2.0 access token when requesting the .default scope using the Client Credentials grant type.
-

10. Logic App Configuration

Create a Consumption-based Azure Logic App named logic-fee-reminder.

Workflow Diagram



1. Recurrence Trigger

- Frequency: Day
- Interval: 1
- Time Zone: India Standard Time (IST)
- Start Time: 09:00 AM daily

2. SQL Query Action (Execute a SQL query (V2))

- Purpose: Retrieve student records with overdue payments.
- Query Condition:
 - PaidAmount < TotalFee
 - AND DueDate < GETDATE()
- SQL Query:

sql

```
SELECT TOP 3 StudentID, Name, Email, TotalFee, PaidAmount, DueDate
FROM dbo.Students
WHERE PaidAmount < TotalFee AND DueDate < GETDATE();
```

3. Outlook Email Action (Send an email (V2))

- Recipient: To: @items('For_each')?['Email']
 - Subject: Fee Payment Reminder - Action Required
 - Email Body Content:
 - Student Name: @{{items('For_each')?['Name']}}
 - Student ID: @{{items('For_each')?['StudentID']}}
 - Total Fee: ₹@{{items('For_each')?['TotalFee']}}
 - Paid Amount: ₹@{{items('For_each')?['PaidAmount']}}
 - Outstanding Fee: ₹@{{sub(items('For_each')?['TotalFee'], items('For_each')?['PaidAmount'])}}
 - Due Date: @{{items('For_each')?['DueDate']}}
-

4. Email Throttling & Delay

- Action: Wait (Delay)
 - Interval: 10 Seconds between successive emails inside the For_each loop to prevent SMTP rate-limit violations.
-

5. Retry Policy

Configure an exponential backoff retry policy on the SQL Query action:

- Retry Policy Type: Exponential
 - Retry Count: 3
 - Minimum Interval: 5 Seconds
 - Maximum Interval: 45 Seconds
-

11. Application Insights

Enable Application Insights during Azure Function App creation or link an existing instance.

Use it to monitor:

- Incoming Request Rate (Live stream of incoming HTTP GET / PUT requests)
 - Server Response Time & Latency (End-to-end execution duration in milliseconds)
 - Unhandled Exceptions & Traces (Runtime errors and database connection issues)
 - Failed Requests & Error Codes (Audit 401 Unauthorized, 404 Not Found, and 500 Internal Errors)
 - Database Dependency Calls (SQL query latency and execution health)
-

12. Testing the Application

Test 1: Payment Status API

- Endpoint: GET /paymentstatus/{studentid}
- Headers: Ocp-Apim-Subscription-Key: <SUBSCRIPTION_KEY>
- Expected Result: Returns HTTP 200 OK with student details (StudentID, Name, TotalFee, PaidAmount, DueDate), calculated OutstandingAmount, and dynamic PaymentStatus (Paid, Overdue, or Partially Paid).

Test 2: Update Fee Payment API (Admin)

- Endpoint: PUT /updatefee/{studentid}
- Payload: {"PaidAmount": <NEW_AMOUNT>}
- Expected Result: Updates the student's PaidAmount in Azure SQL Database and returns HTTP 200 OK ("Fee Status Updated Successfully").

Test 3: Test Unauthorized Access (Zero-Trust Security Check)

- Action: Invoke PUT /updatefee/{studentid} without an Authorization header.
- Expected Result: HTTP 401 Unauthorized — Request is blocked at the APIM gateway before reaching the backend function.

Test 4: Test Authorized Access (Microsoft Entra ID RBAC)

- Action: Invoke PUT /updatefee/{studentid} using:
 - Authorization: Bearer <ENTRA_ID_ACCESS_TOKEN> (Token contains roles: ["Admin"])
 - Subscription Key: Ocp-Apim-Subscription-Key: <SUBSCRIPTION_KEY>
- Expected Result: HTTP 200 OK — APIM validates token signature, audience, and the Admin role claim, then permits the database update.

Test 5: Test API Rate Limiting

- Action: Send more than 10 requests to GET /paymentstatus/{studentid} within a 60-second window.
- Expected Result: HTTP 429 Too Many Requests after exceeding the 10 calls/minute threshold.

Test 6: Test Logic App Automated Reminders

- Action: Trigger the Logic App workflow manually or wait for the daily 9:00 AM IST recurrence trigger.
 - Expected Result: Queries Azure SQL for overdue students (PaidAmount < TotalFee AND DueDate < GETDATE()) and sends formatted HTML reminder emails to the corresponding student email addresses with a 10-second wait between successive emails.
-

13 Project Structure :

Fee Management System/

```
|
|
|— src/
|   |— functions/
|       |— paymentStatus.js    # GET /api/paymentstatus/{studentid} (Student fee & balance
calculation)
|       |— updateFee.js        # PUT /api/updatefee/{studentid} (Admin fee update handler)
|
|— Frontend/
|   |— index.html              # Interactive Web Dashboard UI
|   |— style.css               # Dark-mode dashboard styling
|   |— script.js               # APIM gateway client, DOM renderer & JWT token handler
|
|— APIM/
|   |— payment-status-api-policy.xml # Inbound rate limiting policy (10 calls/min)
|   |— update-fee-api-policy.xml    # Inbound Microsoft Entra ID JWT validation & Admin
RBAC policy
|
|— LogicApp/
|   |— fee-reminder-logicapp.json  # Scheduled daily workflow definition
|   |— FeeReminderLogicAppWorkFlow.png # Logic App designer visual flow
|
|— database/
|   |— database_scripts.txt       # Table schemas, seed records, and SQL queries
|
|— host.json                    # Azure Functions host & Application Insights runtime config
|— local.settings.json          # Local environment variables & SQL connection settings
|— package.json                 # Node.js dependencies (@azure/functions, mssql)
|— README.md                    # Project documentation and run guide``
```

14. Security Considerations

The following sensitive files and credentials must never be committed to public source control:

- local.settings.json (contains local database connection strings & master credentials)
 - node_modules/ (installed package binaries)
 - Microsoft Entra ID Client Secrets
 - Azure API Management Subscription Keys
 - Azure SQL Admin Passwords & Connection Strings
 - OAuth 2.0 Access / Bearer Tokens
-

15. Conclusion

The Fee Management System demonstrates a secure, high-performance, and scalable serverless architecture developed on Microsoft Azure using the Node.js (v4 Model) Azure Functions runtime. The solution seamlessly integrates Azure Functions, Azure SQL Database, Azure API Management (APIM), Microsoft Entra ID (Azure AD), and Azure Logic Apps to deliver reliable student fee inquiry, zero-trust authorized administrator operations, automated background billing reminders, and comprehensive application observability via Application Insights.

16. Appendix: API Testing & Verification Commands

1. Test Payment Status API (Local)

```
curl http://localhost:7071/api/paymentstatus/1
```

2. Test Update Fee API (Local)

```
curl -X PUT \  
-H "Content-Type: application/json" \  
-d '{"PaidAmount": 95000}' \  
http://localhost:7071/api/updatefee/1
```

3. Test Rate Limiting & API Key Authentication (via APIM)

Replace <APIM_NAME>, <API_NAME>, and <SUBSCRIPTION_KEY> with your actual values.

```
for i in {1..12}
do
    curl -i \
        "https://<APIM_NAME>.azure-api.net/<API_NAME>/paymentstatus/1" \
        -H "Ocp-Apim-Subscription-Key: <SUBSCRIPTION_KEY>"
    echo
done
```

PowerShell (Windows):

```
powershell

1..12 | ForEach-Object {
    Invoke-RestMethod -Uri "https://<APIM_NAME>.azure-
api.net/<API_NAME>/paymentstatus/1" `
        -Headers @{ "Ocp-Apim-Subscription-Key" = "<SUBSCRIPTION_KEY>" }
}
```

4. Acquire OAuth 2.0 Access Token from Microsoft Entra ID

```
bash

curl -X POST \
    "https://login.microsoftonline.com/<TENANT_ID>/oauth2/v2.0/token" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -d "client_id=<CLIENT_APPLICATION_ID>" \
    -d "client_secret=<CLIENT_SECRET>" \
    -d "scope=api://<API_APPLICATION_ID>/default" \
    -d "grant_type=client_credentials"
```

5. Test Unauthorized Access (Without Bearer Token)

bash

```
curl -i -X PUT \
  "https://<APIM_NAME>.azure-api.net/<API_NAME>/updatefee/1" \
  -H "Ocp-Apim-Subscription-Key: <SUBSCRIPTION_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"PaidAmount": 110000}'
```

Expected: HTTP 401 Unauthorized

6. Test Secure Admin API (With Bearer Token)

bash

```
curl -X PUT \
  "https://<APIM_NAME>.azure-api.net/<API_NAME>/updatefee/1" \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Ocp-Apim-Subscription-Key: <SUBSCRIPTION_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"PaidAmount": 110000}'
```

Expected Outcome

- ✓ Student fee information retrieved successfully with accurate outstanding balance and dynamic payment status (Paid, Overdue, Partially Paid).
- ✓ Fee records updated exclusively by authenticated administrators via Microsoft Entra ID OAuth 2.0 Bearer tokens and Admin role validation.
- ✓ Overdue fee reminder emails sent automatically to defaulting students on a daily schedule using Azure Logic Apps.
- ✓ API abuse and DDoS prevented via APIM Rate Limiting (10 requests per minute).
- ✓ End-to-end telemetry and request performance monitored in real time through Azure Application Insights.